รายงานโครงงาน ใบเสร็จดี — Offline AI Receipt Analyzer

ระบบวิเคราะห์ใบเสร็จและสรุปค่าใช้จ่ายด้วย AI แบบออฟไลน์บน Raspberry Pi

ผู้จัดทำ: [กรอกชื่อ–นามสกุล / รหัสนักศึกษา] รายวิชา: [กรอก] อาจารย์ที่ปรึกษา: [กรอก]

เอกสารฉบับร่าง วันที่ 19 ตุลาคม 2569 — ต้องเติมภาพอุปกรณ์ ผลทดสอบ Raspberry Pi จริง และข้อมูลผู้จัดทำก่อนส่ง

1. บทนำและปัญหา

การบันทึกค่าใช้จ่ายจากใบเสร็จด้วยมือต้องอ่านรายการและคัดลอกตัวเลขซ้ำ ทำให้เสียเวลาและเกิดข้อผิดพลาดได้ โครงงานนี้ศึกษาการนำ OCR และโมเดลภาษาขนาดเล็กมาช่วยแยกข้อมูลบนอุปกรณ์ภายในเครือข่ายบ้าน โดยให้ผู้ใช้ตรวจแก้ก่อนบันทึกทุกครั้ง เพื่อรักษาความเป็นส่วนตัวและลดการพึ่งพาบริการ AI ภายนอก

2. วัตถุประสงค์

1. พัฒนาระบบเว็บรับภาพใบเสร็จ JPG/PNG ผ่านมือถือใน LAN
2. อ่านภาษาไทยและอังกฤษด้วย Tesseract และใช้ Qwen 2.5 แยกข้อมูลเป็น JSON
3. ตรวจสอบยอดเงินและให้ผู้ใช้แก้ไขก่อนยืนยันบันทึก
4. จัดเก็บใน SQLite และแสดงยอดรายเดือน กราฟหมวดหมู่ และประวัติ
5. ประเมินความถูกต้อง เวลา และการใช้งานออฟไลน์บน Raspberry Pi 4 RAM 4 GB

3. ขอบเขตและข้อจำกัด

ต้นแบบมุ่งใบเสร็จพิมพ์ชัดเจน 2 รูปแบบ มีสินค้าจำนวนไม่มาก รองรับภาพไม่เกิน 1020 ล้านพิกเซล ไม่มีสมาชิก ลายมือ PDF หรือหลายหน้า โมเดลขนาด 0.5 B อาจแยกภาษาไทยผิดและทำงานช้า จึงห้ามบันทึกผล AI อัตโนมัติโดยไม่ตรวจ เมื่อ AI ใช้งานไม่ได้ ระบบเปิดให้ผู้ใช้กรอกเองและแสดงสถานะตามจริง กรณีนี้ไม่ถือว่า AI ผ่านการทดสอบ

4. อุปกรณ์และซอฟต์แวร์

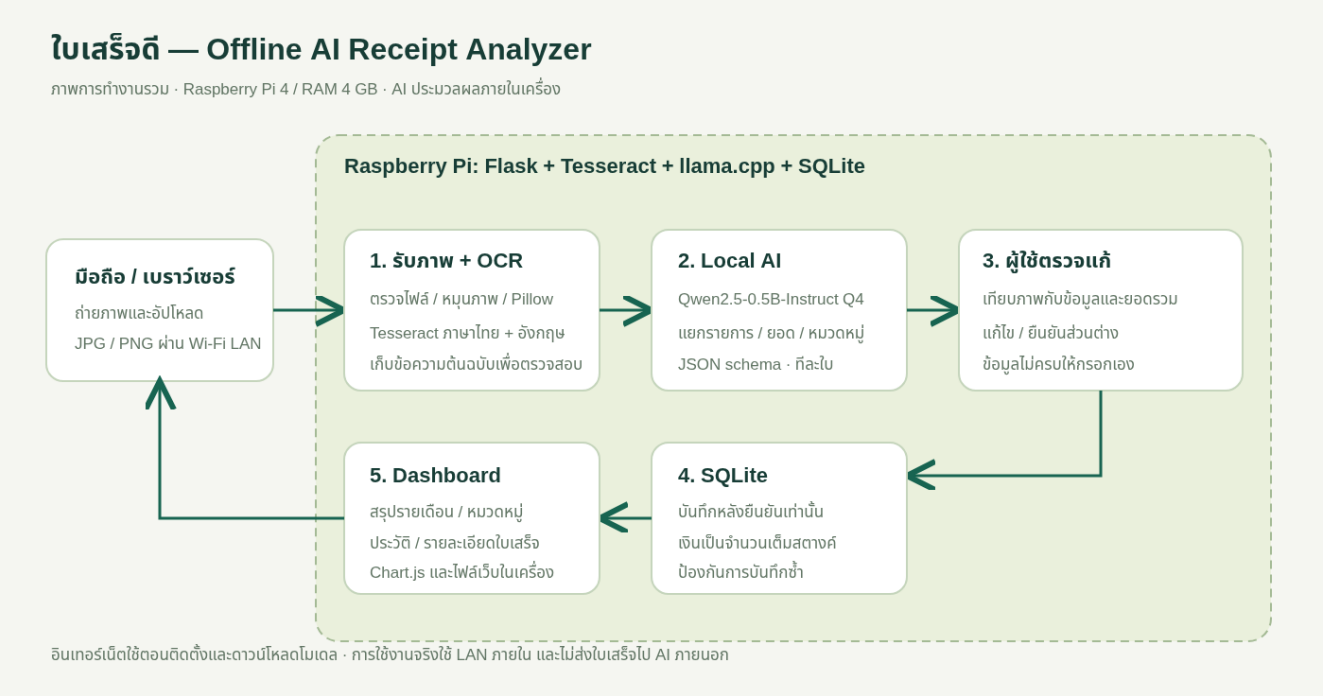
องค์ประกอบ	รายละเอียด
อุปกรณ์เป้าหมาย	Raspberry Pi 4 RAM 4 GB, microSD, แหล่งจ่ายไฟ, ระบบระบายความร้อน
เครือข่าย	เราเตอร์หรือ Wi-Fi LAN และมือถือถ่ายภาพ
OS	Raspberry Pi OS 64-bit / aarch64
เว็บและข้อมูล	Python 3.11+, Flask, Waitress, SQLite

ภาพและ OCR	Pillow, Tesseract, tha + eng
AI	Qwen2.5-0.5B-Instruct GGUF Q4_K_M / llama.cpp
UI	HTML, CSS, JavaScript, Bootstrap 5.3.8, Chart.js 4 5 1

งบประมาณ: [กรอกราคาซื้อจริงและแยกอุปกรณ์ที่มีอยู่แล้ว] ห้ามใช้ราคาโดยประมาณแทนหลักฐานค่าใช้จ่าย

5. การออกแบบและภาพรวมการทำงาน

ภาพประกอบหลัก: architecture.svg



ภาพที่ 1 สถาปัตยกรรมและลำดับการทำงาน

มือถือส่งภาพไป Flask ผ่าน LAN ระบบตรวจไฟล์ หมุนภาพตาม EXIF และทำ OCR ด้วย Tesseract จากนั้นส่งเฉพาะข้อความไป . ที่ loopback ของเครื่องเดียวกัน โมเดลคืนชื่อร้าน วันที่ รายการและหมวดหมู่ด้วย JSON schema ผู้ใช้เทียบกับภาพต้นฉบับ แก้ข้อมูลและยืนยันก่อนบันทึกลง SQLite หน้า อ่านเฉพาะรายการที่ยืนยันแล้ว

สถานะใบเสร็จ: queued → processing → review → confirmed หากวิเคราะห์ล้มเหลวให้เข้า review พร้อมข้อความแจ้ง หากแอปเริ่มใหม่ระหว่างวิเคราะห์จะกู้รายการค้างให้ตรวจเอง เพื่อไม่แสดงว่าผล AI สำเร็จ

ฐานข้อมูล receipts เก็บรูป ข้อความ OCR ผลร่าง สถานะ เวลาประมวลผล ร้าน วันที่ ยอด ส่วนลดและส่วนต่าง ส่วน receipt_items เก็บชื่อ จำนวน ยอดรวมบรรทัดและหมวดหมู่ โดยเชื่อมกับใบเสร็จผ่าน foreign key

6. แผนการพัฒนา

ขั้น	กิจกรรม	ผลส่งมอบ
1	วิเคราะห์เกณฑ์และกำหนดขอบเขต	ผังระบบ / รายการหลักฐาน
2	เตรียม OCR และ llama.cpp	dependencies และสคริปต์ติดตั้ง
3	พัฒนาอัปโหลดและคิววิเคราะห์	รับภาพและเปิดหน้าตรวจแก้
4	ตรวจยอด / transaction / ป้องกันซ้ำ	บันทึกข้อมูลที่ยืนยันแล้ว
5	Dashboard และรองรับมือถือ	กราฟ ตาราง ประวัติ
6	ทดสอบบน cloud และ Pi จริง	TEST_RESULTS.md / PI_ACCEPTANCE.md
7	รายงาน คู่มือ และคลิป	เอกสารประกอบและวิดีโอจริง

Progress ตามเกณฑ์ 27–28 ตุลาคม 2569 งานหลักส่งภายใน 31 ตุลาคม 2569 เป้าหมายทำต้นแบบในวันเดียวเป็นเป้าหมายการพัฒนา ไม่ใช่หลักฐานว่าทดสอบฮาร์ดแวร์ครบแล้ว

7. โค้ดและกลไกสำคัญ

โค้ดฉบับเต็มอยู่ในโฟลเดอร์โปรเจกต์: app.py จัดการ route/คิว, services/ocr.py อ่านภาพ, services/llm.py ติดต่อโมเดลภายในเครื่อง, services/validation.py ตรวจข้อมูล และ services/database.py สร้างฐานข้อมูล

ตัวอย่างการเก็บเงินที่หลีกเลี่ยง binary float:

```
amount = Decimal(str(value).strip())
if amount != amount.quantize(Decimal('0.01')):
    raise ValueError
return int(amount * 100)
```

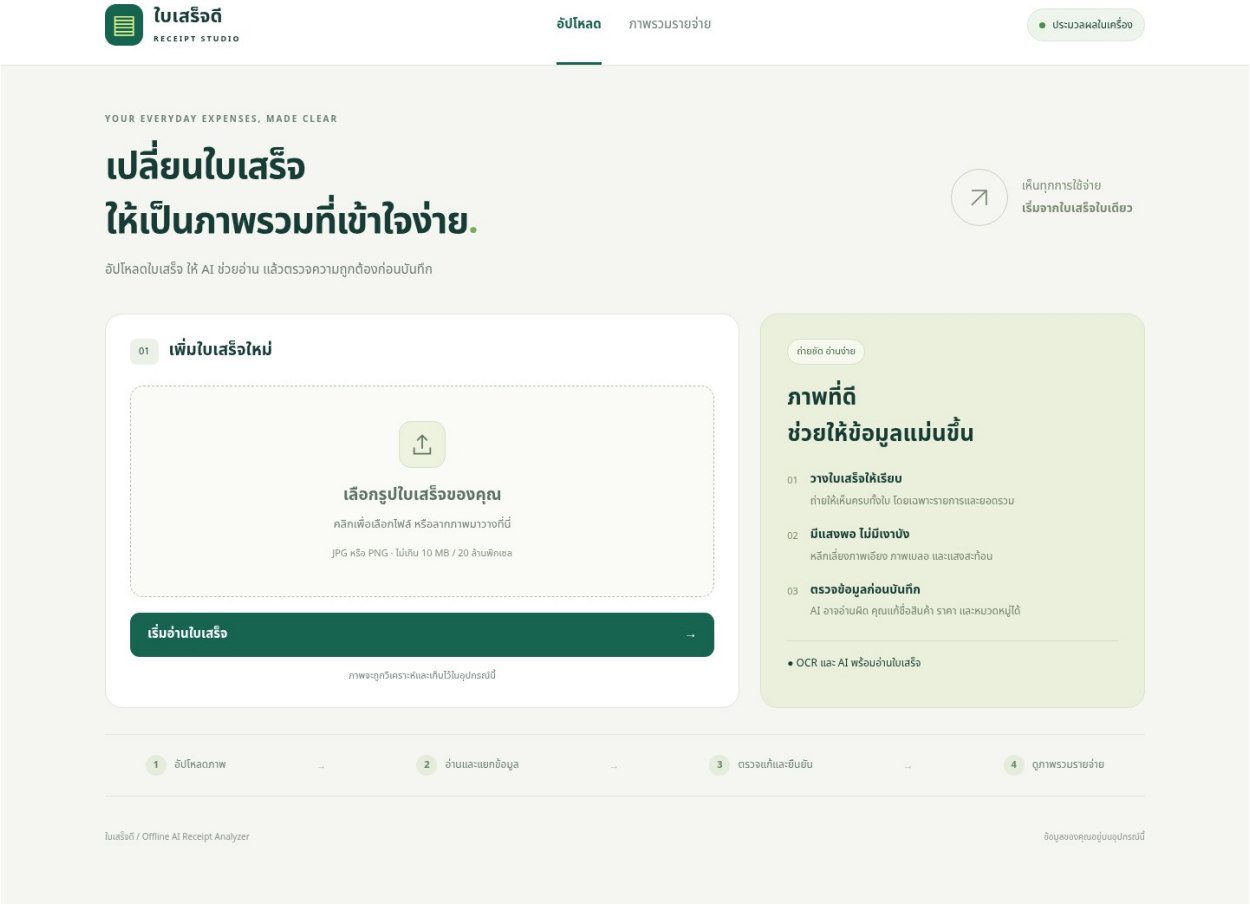
ยอดรวมที่คาด = ผลรวมยอดสินค้าทุกบรรทัด + ส่วนลด/ค่าบริการ ผู้ใช้ต้องยืนยันส่วนต่างถ้ามี ระบบไม่คูณยอดรวมบรรทัดด้วยจำนวนสินค้าซ้ำ และบันทึกใบเสร็จกับรายการใน transaction เดียวภายใต้ BEGIN IMMEDIATE เพื่อป้องกันกดยืนยันซ้ำพร้อมกัน

LLAMA_URL รับเฉพาะ loopback, requests ไม่ส่งการวิเคราะห์ผ่าน , หน้าเว็บใช้ไฟล์ UI จากเครื่อง, มี CSRF token และ escape ข้อความก่อนแสดง อย่างไรก็ตามรุ่นนี้ไม่มีระบบสมาชิก จึงกำหนดใช้งานใน LAN ที่ไว้ใจได้

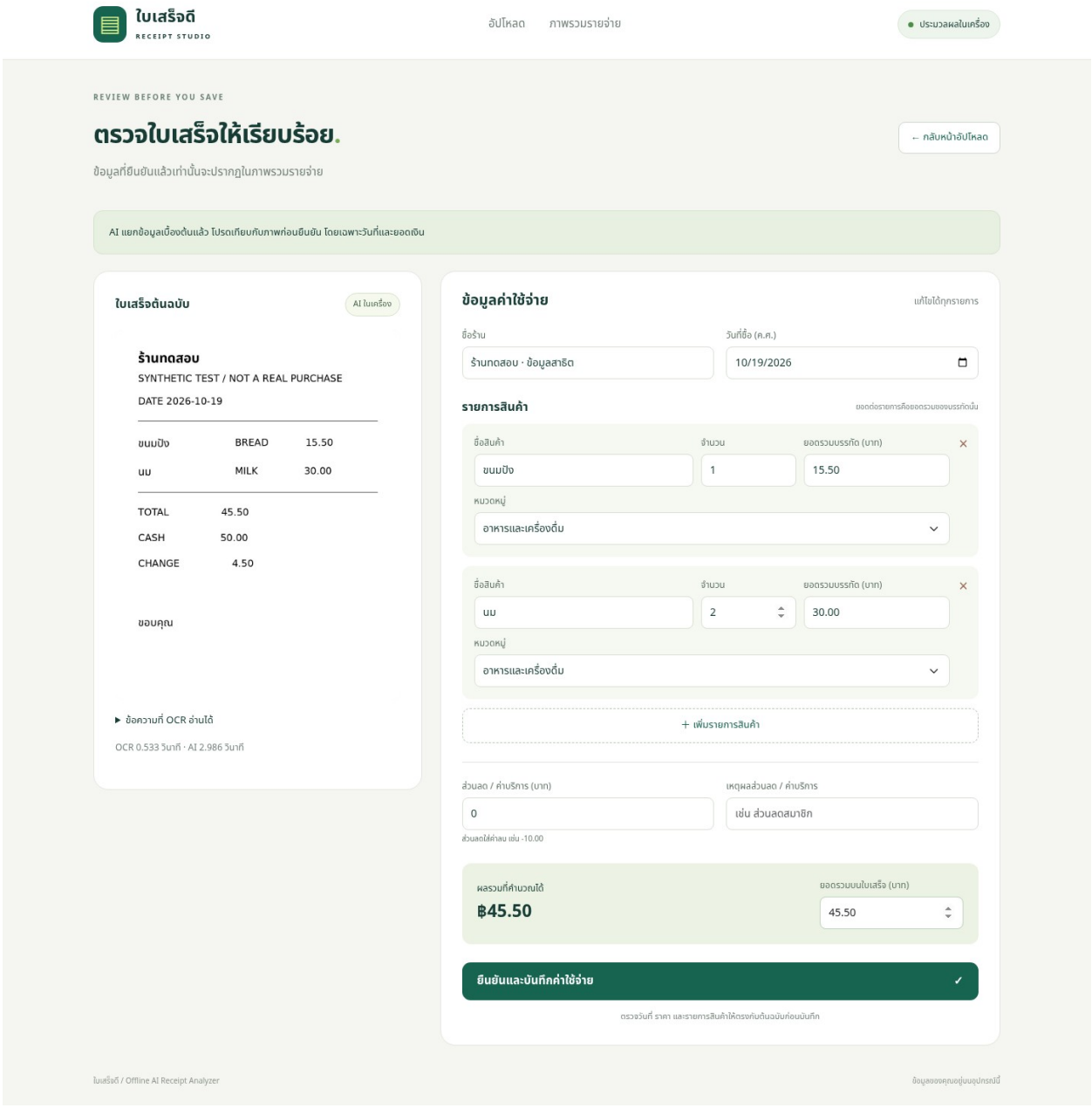
8. ผลการทดลองและหลักฐาน

ผลล่าสุดที่ตรวจแล้วจริงอยู่ใน TEST_RESULTS.md และ evidence/browser-results.json ภาพหน้าจอ evidence/01-upload.png, 02-review.png และ 03-dashboard.png มาจากเว็บจริงโดยใช้ใบเสร็จสังเคราะห์ที่ติดป้ายว่าเป็นข้อมูลสาธิต

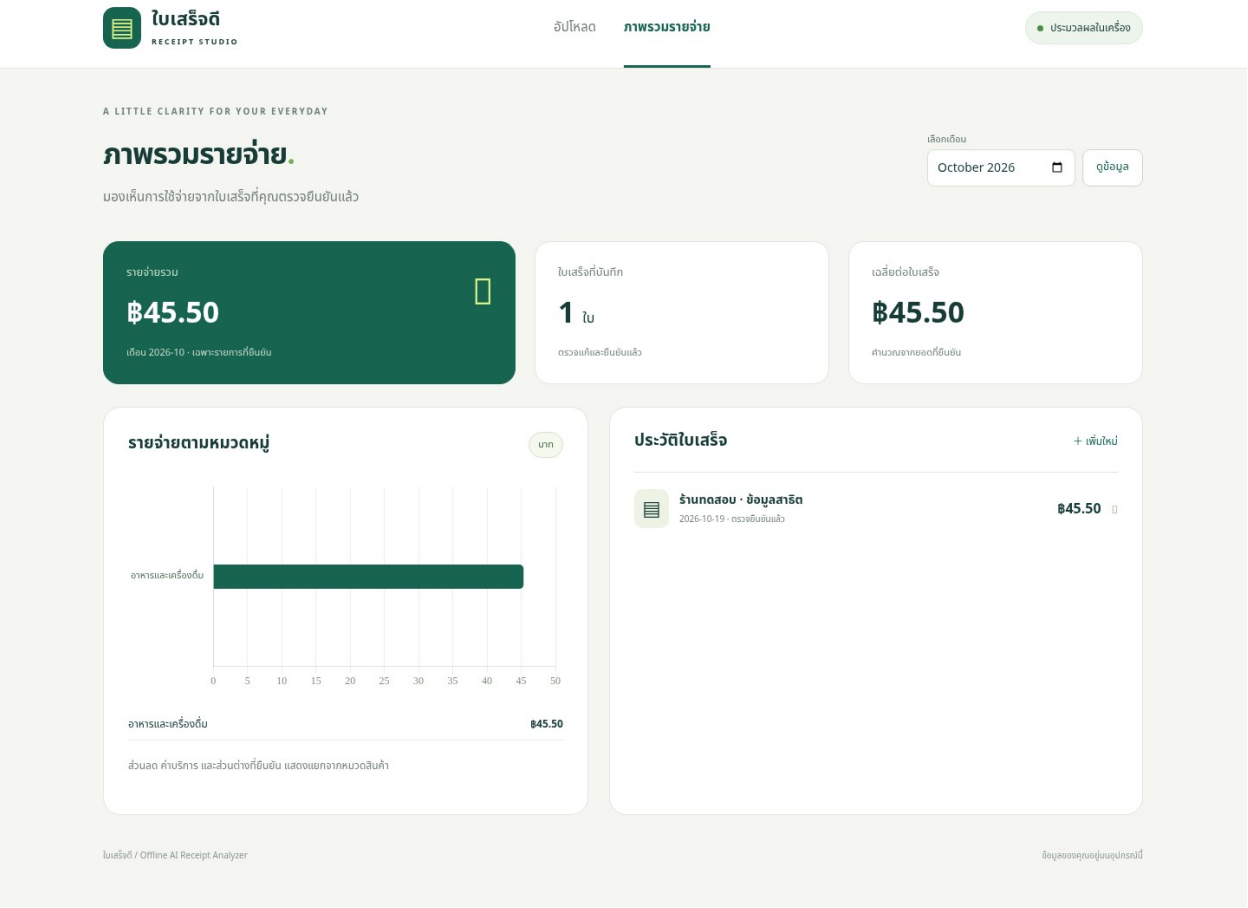
ผล cloud ตรวจการอัปโหลด OCR ไทย/อังกฤษ ตรวจแก้ บันทึก Dashboard การคำนวณเงิน ป้องกันซ้ำ และการโหลด UI โดยปิดคำขอภายนอก ผลนี้ไม่ใช่การวัดประสิทธิภาพบน Raspberry Pi และภาพสังเคราะห์ไม่ใช้สรุปความแม่นยำของใบเสร็จจริง



หน้าอัปโหลด — ข้อมูลสาธิตจาก browser smoke test



หน้าตรวจแก้ — ข้อมูลสาธิตจาก browser smoke test



Dashboard — ข้อมูลสาธิตจาก browser smoke test

สถานะ Local LLM: ดาวน์โหลด Qwen GGUF และตรวจ SHA 256ผ่าน คอมไพล์ llama.cpp และเรียก inference จริงบน x8 6 _6 4 ได้ ภาพสังเคราะห์หนึ่งใบใช้ OCR ประมาณ 0 . 4 1 วินาทีและ AI 4.59 วินาที โมเดลคืน JSON ได้แต่ชื่อร้าน ราคาหนึ่งรายการ และหมวดหมู่ผิด ต้องแก้ก่อนยืนยัน ผลนี้ไม่ใช่ความเร็วบน Pi และยังใช้สรุปความแม่นยำใบเสร็จจริงไม่ได้ ผลดิบเก็บใน evidence/model-results.json

ผล Raspberry Pi: [ยังไม่ทดสอบ — เติมตาราง PI_ACCEPTANCE.md จำนวน 1 0 ใบ 2 รูปแบบ พร้อมเวลา OCR/AI รายใบ ความถูกต้องของชื่อ ราคาและยอด และจำนวนช่องที่ผู้ใช้แก้]

แนบภาพ Pi จริง: [รอผู้จัดทำถ่าย] การทดสอบตัดอินเทอร์เน็ตและรีบูต Pi: [รอทดสอบจริง]

9. นวัตกรรมและการประยุกต์

จุดเด่นที่ออกแบบไว้คือการรวม OCR ภาษาไทยและโมเดลขนาดเล็กบน Edge device กับกระบวนการให้ผู้ใช้ตรวจยอดจากหลักฐานก่อนบันทึก โดยแสดงส่วนต่างอย่างโปร่งใส แนวคิดนี้เป็นการประยุกต์เทคโนโลยีที่มีอยู่ จึงไม่อ้างว่าเป็นสิ่งที่ไม่เคยมีในตลาด

งานเปรียบเทียบที่ต้องเติม: [อย่างน้อย 2 แนวทาง เช่น กรอกเองและบริการสแกนใบเสร็จ โดยอ้างแหล่งข้อมูลจริง] เปรียบเทียบอินเทอร์เน็ต การส่งข้อมูลออก ค่าใช้จ่าย การแก้ไขข้อมูล และเวลาที่วัดได้

10. สรุปและแนวทางต่อยอด

ได้พัฒนาซอฟต์แวร์ต้นแบบและทดสอบ OCR → Local AI → ผู้ใช้ตรวจแก้ → บันทึก → Dashboard บน cloud ครบกระบวนการ มีหลักฐานการทดสอบอัตโนมัติและโมเดลจริง ความแม่นยำของ AI ยังจำกัดและต้องมีผู้ใช้ตรวจแก้ การทดลองใบเสร็จจริงบน Pi รวมการตัดอินเทอร์เน็ตและรีบูตยังเป็นเงื่อนไขก่อนสรุปว่าพร้อมส่งงาน

แนวทางต่อยอด ได้แก่ ชุดทดสอบใบเสร็จจริงที่หลากหลายขึ้น ปรับการจัดบรรทัด OCR รองรับใบเสร็จยาว แก้ข้อมูลหลังยืนยัน และวัดประสบการณ์ผู้ใช้ด้วยผู้ทดสอบภายนอก

11. แหล่งอ้างอิง

- เกณฑ์ประเมิน Project IoT จากไฟล์ที่ผู้ใช้ให้ (2 0 คะแนน)
- Flask: https://flask.palletsprojects.com/
- Tesseract: https://tesseract-ocr.github.io/
- llama.cpp: https://github.com/ggml-org/llama.cpp
- Qwen GGUF: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF
- SQLite: https://www.sqlite.org/
- Bootstrap / Chart.js: ดู THIRD_PARTY.md